PROCEDIMIENTO DE IMPLEMENTACIÓN DEL SOFTWARE

PROYECTO: Traduce Señas

INTEGRANTES:
Luis Alejandro Duarte Aldana
Juan Mauricio Suaza Solorzano
Juan Camilo Fierro Esquivel

SERVICIO NACIONAL DE APRENDIZAJE – SENA
Análisis y Desarrollo De Software

2026

Tabla de contenido

Introducción 3
Alcance 3
Arquitectura de despliegue 4
Requisitos previos 4
Procedimiento de implementación 5
Paso 1. Preparar el servidor 5
Paso 2. Obtener el código 5
Paso 3. Configurar las variables de entorno 5
Paso 4. Construir y levantar el backend 5
Paso 5. Verificar el backend 6
Paso 6. Publicar la versión web y de escritorio 6
Paso 7. Distribuir la aplicación móvil 6
Paso 8. Crear el usuario administrador 6
Paso 9. Cargar el contenido inicial 6
Pruebas de aceptación de la implementación 7
Plan de reversión 7
Operación posterior 7
Conclusiones 8
Referencias 9

Introducción

El procedimiento de implementación describe, paso a paso, cómo poner Traduce Señas en funcionamiento en un entorno de producción o de pruebas: qué se necesita, en qué orden se instalan los componentes, cómo se verifica que todo funciona y cómo se regresa a la versión anterior si algo falla.

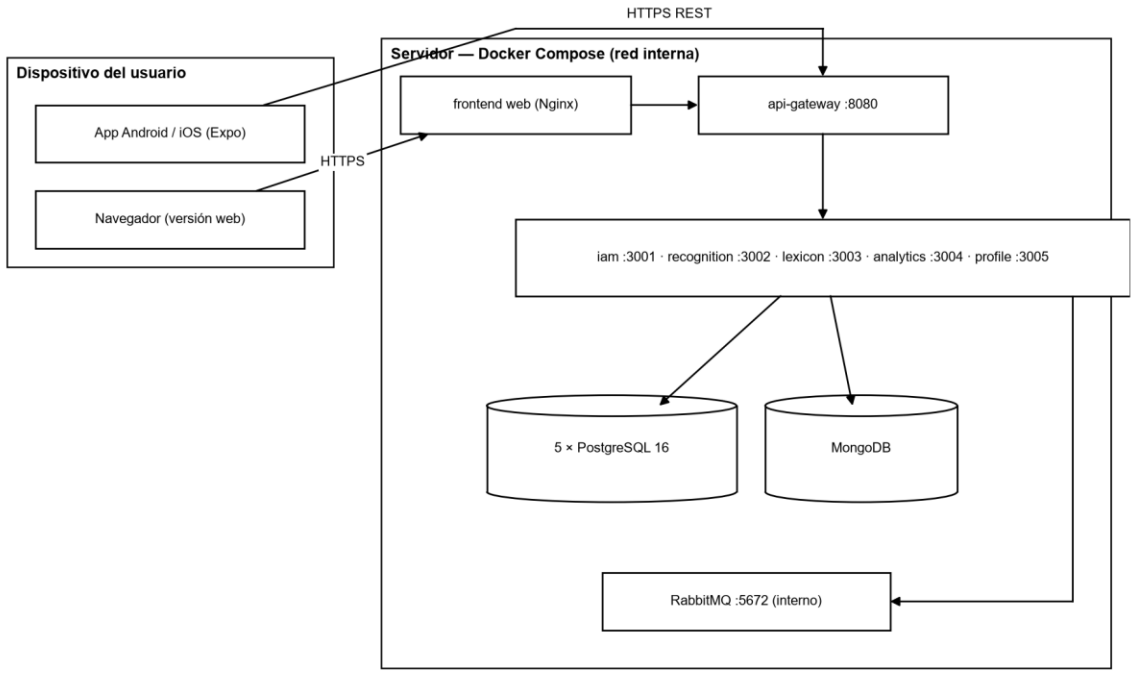
El procedimiento está pensado para el administrador del sistema o el integrante del equipo responsable del despliegue, y parte de un servidor limpio con acceso a internet.

Alcance

Cubre el despliegue del backend completo (API Gateway, cinco microservicios, bases de datos y RabbitMQ), la publicación de la versión web y de escritorio, y la distribución de la aplicación móvil. No cubre la configuración de dominios ni de certificados de una organización específica, que se mencionan como requisitos previos.

Arquitectura de despliegue

Figura 1
Diagrama de despliegue del sistema



Nota. Elaboración propia.

Solo el API Gateway (puerto 8080) y el servidor web quedan expuestos a la red. Los microservicios (3001 a 3005), las bases de datos y RabbitMQ solo son accesibles dentro de la red interna de Docker; los puertos de las bases de datos se publican únicamente en 127.0.0.1 para tareas de administración.

Requisitos previos

Tabla 1
Requisitos del servidor

Recurso	Mínimo	Recomendado
Procesador	2 vCPU	4 vCPU
Memoria RAM	4 GB	8 GB
Disco	20 GB	50 GB SSD
Sistema operativo	Linux de 64 bits (Ubuntu 22.04 LTS) o Windows con Docker Desktop	Ubuntu 24.04 LTS
Software	Docker 24+, Docker Compose 2.20+, Git 2.40+	Igual, con Node.js 20 LTS para tareas de mantenimiento

Recurso	Mínimo	Recomendado
Red	Puertos 80/443 y 8080 abiertos hacia internet	Proxy inverso con HTTPS delante del gateway

Tabla 2
Información que debe tenerse antes de empezar

Dato	Uso
JWT_SECRET (32 caracteres o más)	Firma de los tokens de sesión; generar con «openssl rand -base64 48»
POSTGRES_PASSWORD	Contraseña de las cinco bases PostgreSQL
MONGO_ROOT_PASSWORD	Contraseña de MongoDB
RABBITMQ_PASSWORD	Contraseña de RabbitMQ
EMAIL_USER / EMAIL_PASS	Cuenta SMTP para enviar códigos de verificación y recuperación
CORS_ORIGINS	Dominios web autorizados, separados por comas
Certificado TLS	HTTPS obligatorio en producción

Nota. Las contraseñas no deben contener los caracteres @ : / ? # porque se incluyen dentro de las URL de conexión.

Procedimiento de implementación

Paso 1. Preparar el servidor

```
sudo apt update && sudo apt upgrade -y
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER      # cerrar sesión y volver a entrar
docker --version && docker compose version
```

Paso 2. Obtener el código

```
git clone https://github.com/mauro-1508/backendTS.git backend
git clone https://github.com/tadeo77789/frontendTS.git frontend
cd backend && git checkout main   # o la etiqueta de la versión a desplegar
```

Paso 3. Configurar las variables de entorno

```
cp .env.example .env
nano .env    # completar JWT_SECRET, POSTGRES_PASSWORD, MONGO_ROOT_PASSWORD,
             # RABBITMQ_PASSWORD, EMAIL_USER, EMAIL_PASS y CORS_ORIGINS
```

Docker Compose se detiene con un mensaje de error si falta alguna variable obligatoria; esto evita arrancar con contraseñas predeterminadas.

Paso 4. Construir y levantar el backend

```
docker compose up --build -d
docker compose ps     # todos los contenedores deben quedar en estado
running/healthy
```

Al arrancar, cada servicio aplica sus migraciones SQL (carpeta db/ de cada servicio) sobre su propia base de datos.

Paso 5. Verificar el backend

```
curl http://localhost:8080/health                # gateway
docker compose logs -f api-gateway    # revisar errores
# Panel de RabbitMQ (solo desde el servidor): http://127.0.0.1:15672
```

Paso 6. Publicar la versión web y de escritorio

```
cd ../frontend
# Ajustar la URL del gateway en src/app/config/api.config.ts (por ejemplo
https://api.midominio.co)
docker build -t traduce-senas-web .
docker run -d --name traduce-web -p 80:80 traduce-senas-web
```

Paso 7. Distribuir la aplicación móvil

1. Configurar en api.config.ts la URL pública del gateway (con HTTPS).
2. Generar el instalador con EAS Build: «npx eas build --platform android» (AAB/APK) o «--platform ios» (IPA).
3. Para pruebas piloto, distribuir el APK directamente o por Expo Go; para el público, publicar en Google Play y App Store.

Paso 8. Crear el usuario administrador

Registrar una cuenta desde la aplicación, verificar el correo y asignarle el rol ADMIN con un administrador existente (POST /api/iam/users/:id/roles) o, en la primera instalación, insertando la relación en la tabla user_roles de la base iam.

Paso 9. Cargar el contenido inicial

- Importar las plantillas de señas del alfabeto y de las palabras (rol ADMIN, ruta /api/sign-templates).
- Verificar que el alfabeto aparece en la aplicación con sus 27 letras (A–Z y Ñ).
- Revisar las categorías y señas publicadas en el léxico.

Pruebas de aceptación de la implementación

Tabla 3
Lista de verificación posterior al despliegue

N.º	Verificación	Resultado esperado
1	GET /health en el gateway	200 OK
2	Registro de un usuario nuevo	Correo con código recibido; cuenta activa tras verificar
3	Inicio de sesión	Token recibido; panel principal visible
4	Traducción con la cámara	Texto mostrado y guardado en el historial
5	Logro «Primera traducción»	Aparece en «Mis logros» (evento procesado por profile-service)
6	Estadísticas con rol ADMIN	Gráficas con el evento de la prueba 4
7	Acceso a estadísticas con rol USER	403 Prohibido
8	Versión web en navegador de escritorio	Interfaz adaptada y cámara funcional

Nota. El flujo de las pruebas 2 a 6 se ejecutó con Docker en el entorno local del equipo el 5 de octubre de 2026 con resultado satisfactorio.

Plan de reversión

1. Antes de cada despliegue, crear una copia de seguridad: «docker compose exec postgres-iam pg_dump …» por cada base y «mongodump» para MongoDB.
2. Etiquetar la versión desplegada en Git (por ejemplo, v1.2.0).
3. Si la verificación falla: «docker compose down», volver a la etiqueta anterior con «git checkout v1.1.0» y ejecutar «docker compose up --build -d».
4. Si la migración modificó datos, restaurar las copias con «psql < respaldo.sql» y «mongorestore».
5. Registrar el incidente en el documento de acciones correctivas.

Operación posterior

Tabla 4
Tareas de operación

Tarea	Frecuencia	Comando o acción
Copia de seguridad de bases de datos	Diaria	pg_dump por servicio y mongodump, conservados 7 días
Revisión de registros	Diaria	docker compose logs --since 24h
Actualización de dependencias	Mensual	npm audit y actualización controlada en rama
Revisión de precisión del reconocimiento	Mensual	Comparar traducciones no reconocidas por seña
Prueba de restauración	Trimestral	Restaurar una copia en un entorno de pruebas

Conclusiones

Gracias a Docker Compose, la implementación del backend completo se reduce a configurar un archivo .env y ejecutar un comando, y la versión web se entrega como un contenedor Nginx. La lista de verificación y el plan de reversión permiten desplegar con seguridad y volver atrás en minutos si algo falla.

Referencias

Broadcom. (2026). *RabbitMQ documentation*. https://www.rabbitmq.com/docs

Docker Inc. (2026). *Docker Compose documentation*. https://docs.docker.com/compose/

Expo. (2026). *Expo documentation*. https://docs.expo.dev/

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.ª ed.). O'Reilly Media.

OWASP Foundation. (2021). *OWASP Top 10: 2021*. https://owasp.org/Top10/

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.